Agentic RAG with LangGraph: Discovery-to-Action Strategy

This notebook implements an intelligent AI assistant using Python, LangChain, LangGraph, and ChromaDB, focusing on the Discovery-to-Action (DTA) framework for Retrieval-Augmented Generation (RAG).

1. Environment Setup: Install Libraries

First, we'll install all the necessary Python libraries for LangChain, LangGraph, ChromaDB, PDF parsing, and the OpenAI API. We'll also need `unstructured` and `tiktoken` for document processing.

Switch to Free Alternatives: HuggingFace Embeddings and HuggingFace LLM

Since an OpenAI `RateLimitError` indicates a billing or quota issue, and Google Generative AI quotas are exhausted, we will switch to other free alternatives to proceed with the notebook.

- **HuggingFace Embeddings**: We will use `sentence-transformers` for generating embeddings.
- **HuggingFace LLM**: We will use a model from HuggingFace Hub via `langchain-huggingface` for the LLM.

You will need to obtain a **HuggingFace API token** from HuggingFace Settings and add it to your Colab secrets, named `HF_TOKEN`.

```
%%capture
!pip install -U sentence-transformers langchain-huggingface
```

```
%%capture
!pip install -U langchain langchain-community langchain-chroma langgraph pypdf
```

2. Configure API Keys

We will use OpenAI for embeddings and the LLM. Please make sure your OpenAI API key is added to Colab secrets as `OPENAI_API_KEY`.

```
# Used to securely store your API key
from google.colab import userdata
import os

# Commenting out OpenAI API key loading as we are switching to HuggingFace LLM
# os.environ["OPENAI_API_KEY"] = userdata.get('OPENAI_API_KEY')

# if os.environ["OPENAI_API_KEY"] is None:
#     raise ValueError("OPENAI_API_KEY not found in Colab secrets. Please add i

print("OpenAI API Key loading skipped as we are using HuggingFace LLM.")
```

```
OpenAI API Key loading skipped as we are using HuggingFace LLM.
```

```
# Used to securely store your API key
from google.colab import userdata
import os

# Uncommenting GOOGLE_API_KEY loading to use Google Generative AI.
os.environ["GOOGLE_API_KEY"] = userdata.get('GOOGLE_API_KEY')

if os.environ["GOOGLE_API_KEY"] is None:
    raise ValueError("GOOGLE_API_KEY not found in Colab secrets. Please add it.

print("Google API Key loaded successfully.")
```

```
Google API Key loaded successfully.
```

3. Discovery Phase: Document Loading, Chunking, and Embedding

We'll load a sample PDF document, split it into manageable chunks, and generate embeddings to store in a persistent ChromaDB instance. For demonstration purposes, I will download a sample PDF, but you can replace the URL with your desired document.

Note: If you want to use your own PDF, upload it to your Colab environment and update the `pdf_path` variable.

```
import requests
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
# from langchain_openai import OpenAIEmbeddings # Old import
from langchain_huggingface.embeddings import HuggingFaceEmbeddings # New, upda
from langchain_chroma import Chroma
import os

# Download a sample PDF (e.g., an AI Ethics paper)
pdf_url = "https://arxiv.org/pdf/2307.03175.pdf" # Example: 'Ethics of AI: Risk
pdf_path = "./ai_ethics.pdf"

# Check if the PDF already exists to avoid re-downloading
if not os.path.exists(pdf_path):
```

```
    print(f"Downloading {pdf_url} to {pdf_path}...")
    response = requests.get(pdf_url)
    response.raise_for_status() # Raise an exception for HTTP errors
    with open(pdf_path, "wb") as f:
        f.write(response.content)
    print("Download complete.")
else:
    print(f"PDF already exists at {pdf_path}. Skipping download.")

# Load the document
loader = PyPDFLoader(pdf_path)
docs = loader.load()
print(f"Loaded {len(docs)} pages from {pdf_path}.")

# Chunk the document into ~500-character segments
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50,
    add_start_index=True,
)
chunks = text_splitter.split_documents(docs)
print(f"Split into {len(chunks)} chunks.")

# Initialize HuggingFace Embeddings
# embeddings = OpenAIEmbeddings(model="text-embedding-ada-002") # Old initializ
embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2") # New initial

# Create a persistent ChromaDB instance
persistent_directory = "./chroma_db"
vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    persist_directory=persistent_directory
)

print(f"ChromaDB created and data persisted to {persistent_directory}.")
print(f"Number of vectors in ChromaDB: {vectorstore._collection.count()}")
```

```
PDF already exists at ./ai_ethics.pdf. Skipping download.
Loaded 18 pages from ./ai_ethics.pdf.
Split into 134 chunks.
Loading weights: 100%                                    103/103 [00:00<00:00, 2855.06it/s]
ChromaDB created and data persisted to ./chroma_db.
Number of vectors in ChromaDB: 268
```

```
# Uncommenting GOOGLE_API_KEY loading to use Google Generative AI.
from google.colab import userdata
import os

os.environ["GOOGLE_API_KEY"] = userdata.get('GOOGLE_API_KEY')

if os.environ["GOOGLE_API_KEY"] is None:
    raise ValueError("GOOGLE_API_KEY not found in Colab secrets. Please add it

print("Google API Key loaded successfully.")
```

```
Google API Key loaded successfully.
```

```
from langgraph.graph import StateGraph, END

# Define the graph
workflow = StateGraph(AgentState)

# Add nodes for the agent and the tool executor
workflow.add_node("agent", agent_node)
workflow.add_node("tool", tool_node)

# Set the entry point
workflow.set_entry_point("agent")

# Define edges
# If the agent decides to call a tool, route to the 'tool' node
workflow.add_conditional_edges(
    "agent",
    should_continue,
    {
        "call_tool": "tool",
        "end": END
    }
)

# After the tool is executed, route back to the 'agent' node for response gener
workflow.add_edge('tool', 'agent')

# Compile the graph into a runnable agent
app = workflow.compile()

print("LangGraph workflow built and compiled!")
```

```
LangGraph workflow built and compiled!
```

4. Create a LangChain Tool for Knowledge Base Search

Now we'll define a LangChain tool that queries our ChromaDB vector store. This tool will allow our agent to retrieve relevant context when needed.

```
from langchain.tools import tool

@tool
def knowledge_base_search(query: str) -> str:
    """
    Searches the vector database for relevant information based on the query.
    Returns a string containing the concatenated content of relevant documents
    """
    print(f"\n--- Invoking knowledge_base_search with query: '{query}' ---")
    results = vectorstore.similarity_search(query, k=4) # Retrieve top 4 simila
    # Concatenate the content of the relevant documents into a single string
    context = "\n\n".join([doc.page_content for doc in results])
    print(f"--- Retrieved {len(results)} documents. Context length: {len(contex
```

```
        return context

    # Test the tool
    # print(knowledge_base_search("What are the main risks of bias in AI?"))
```

5. Technical Phase: Define LangGraph State

We'll define a Pydantic `State` class using `TypedDict` to track the conversation history, user input, and tool outputs within our LangGraph workflow.

```
    from typing import List, Annotated, TypedDict
    from langchain_core.messages import BaseMessage, HumanMessage, AIMessage

    class AgentState(TypedDict):
        """
        Represents the state of our agent throughout the conversation.
        """
        input: str # The user's input query
        chat_history: Annotated[List[BaseMessage], lambda x, y: x + y] # Accumulat
        tool_output: str # Output from the knowledge_base_search tool
        final_answer: str # The final answer from the LLM
```

6. Initialize the Language Model (LLM)

We'll use OpenAI's Chat Model for our agent's reasoning capabilities.

```
    from langchain_google_genai import ChatGoogleGenerativeAI
    import os

    # Ensure GOOGLE_API_KEY is loaded from Colab secrets
    if os.environ.get("GOOGLE_API_KEY") is None:
        from google.colab import userdata
        os.environ["GOOGLE_API_KEY"] = userdata.get('GOOGLE_API_KEY')
        if os.environ["GOOGLE_API_KEY"] is None:
            raise ValueError("GOOGLE_API_KEY not found in Colab secrets. Please ad

    # Initialize the Google Generative AI Chat Model (gemini-2.5-flash for general
    llm = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0.1, conver
    print("LLM initialized using Google Generative AI (gemini-2.5-flash).")
```

```
LLM initialized using Google Generative AI (gemini-2.5-flash).
```

7. Define Agent and Tool Nodes

We'll create the agent node, which will use the LLM to decide whether to answer directly or use the `knowledge_base_search` tool. We'll also define a tool node that simply

executes the tool.

```
from langchain_core.messages import ToolMessage
from langchain_core.runnables import RunnablePassthrough
from langchain_core.tools import tool as langchain_tool
from langgraph.prebuilt import ToolNode

# Combine our custom tool with any other tools if necessary
# For this project, we only have one custom tool
langchain_tools = [knowledge_base_search]

# Define the Tool Node - explicitly tell it where to find messages in the state
tool_node = ToolNode(langchain_tools, messages_key="chat_history")

# Define the Agent Node
def agent_node(state: AgentState):
    """
    The agent node: decides whether to call a tool or respond directly.
    """
    print("\n--- Entering Agent Node ---")
    messages = state["chat_history"]

    # If there's tool output, add it to chat history for LLM to consider
    if state.get("tool_output"):
        messages.append(ToolMessage(content=state["tool_output"], tool_call_id=

    # Craft a system prompt to enforce grounding
    system_prompt = (
        "You are an AI research assistant. Your primary goal is to provide accu
        "If you use the 'knowledge_base_search' tool, you MUST use the informat
        "If the 'knowledge_base_search' tool provides insufficient context to a
        "you MUST respond with 'I don't know' and explain why the information w
        "For questions that do not require external knowledge (e.g., simple ari
        "Always be truthful and do not hallucinate information."
    )

    # Bind the tools to the LLM
    agent_runnable = llm.bind_tools(langchain_tools)

    # Invoke the agent with chat history and system prompt
    response = agent_runnable.invoke([HumanMessage(content=system_prompt)] + me

    # Update chat history and decide next step
    updated_history = messages + [response]
    return {"chat_history": updated_history}
```

8. Implement Conditional Edge (`should_continue`)

This function will determine whether the agent needs to call a tool or can respond directly based on the LLM's output.

```
from langgraph.graph import StateGraph, END

def should_continue(state: AgentState):
```

```
    """
    Conditional edge function: decides whether to continue tool calls or end.
    """
    print("\n--- Deciding Next Step ---")
    messages = state["chat_history"]
    last_message = messages[-1]

    # If the last message is a tool call, we need to execute the tool
    if last_message.tool_calls:
        print("Decision: CALL_TOOL")
        return "call_tool"
    else:
        # Otherwise, the LLM has provided a final answer
        print("Decision: END (Agent responded directly)")
        return "end"
```

9. Build and Compile the LangGraph Workflow

Now, we'll instantiate the `StateGraph`, add our defined agent and tool nodes, set the entry point, and define the edges to control the flow, including the conditional edge for dynamic routing. Finally, we'll compile it into an executable `AgentExecutor`.

```
from langgraph.graph import StateGraph, END

# Define the graph
workflow = StateGraph(AgentState)

# Add nodes for the agent and the tool executor
workflow.add_node("agent", agent_node)
workflow.add_node("tool", tool_node)

# Set the entry point
workflow.set_entry_point("agent")

# Define edges
# If the agent decides to call a tool, route to the 'tool' node
workflow.add_conditional_edges(
    "agent",
    should_continue,
    {
        "call_tool": "tool",
        "end": END
    }
)

# After the tool is executed, route back to the 'agent' node for response gener
workflow.add_edge('tool', 'agent')

# Compile the graph into a runnable agent
app = workflow.compile()

print("LangGraph workflow built and compiled!")
```

```
LangGraph workflow built and compiled!
```

10. Action Phase: Testing the Agent

Now that our agent is compiled, let's test its capabilities with different types of queries to verify its routing logic and factual grounding. We'll perform a retrieval test and a logic test.

Retrieval Test: Document-Specific Question

We'll ask a question that requires knowledge base search to answer. Observe if the `knowledge_base_search` tool is invoked and if the answer is grounded in the retrieved context.

```
# We no longer need to list Gemini models as we are using HuggingFace LLM.
# import google.generativeai as genai

# print("Listing available Gemini models...")
# for m in genai.list_models():
#   if "generateContent" in m.supported_generation_methods:
#     print(m.name)

print("\n--- Running Retrieval Test ---")
query_retrieval = "What are the three main risks of bias mentioned in the docu

# Initialize state for the retrieval query
initial_state_retrieval = {"input": query_retrieval, "chat_history": [HumanMes

# Run the agent
final_state_retrieval = None
for s in app.stream(initial_state_retrieval):
    print(s)
    if not final_state_retrieval: # Capture the final state
        for key, value in s.items():
            if key != '__end__':
                final_state_retrieval = value

print(f"\nAgent's Final Response (Retrieval Test): {final_state_retrieval['cha
```

```
--- Running Retrieval Test ---

--- Entering Agent Node ---

--- Deciding Next Step ---
Decision: CALL_TOOL
{'agent': {'chat_history': [HumanMessage(content='What are the three main

--- Invoking knowledge_base_search with query: 'three main risks of bias'
--- Retrieved 4 documents. Context length: 1080 ---
{'tool': {'chat_history': [ToolMessage(content='as rotating the Dracaena)

--- Entering Agent Node ---
```

```
--- Deciding Next Step ---
Decision: END (Agent responded directly)
{'agent': {'chat_history': [HumanMessage(content='What are the three main

Agent's Final Response (Retrieval Test):
```

Logic Test: Simple Question

We'll ask a question that does *not* require database lookup. Observe if the agent correctly bypasses the `knowledge_base_search` tool and responds directly.

```
print("\n--- Running Logic Test ---")
query_logic = "What is 2+2?"

# Initialize state for the logic query
initial_state_logic = {"input": query_logic, "chat_history": [HumanMessage(cont

# Run the agent
final_state_logic = None
for s in app.stream(initial_state_logic):
    print(s)
    if not final_state_logic: # Capture the final state
        for key, value in s.items():
            if key != '__end__':
                final_state_logic = value

print(f"\nAgent's Final Response (Logic Test): {final_state_logic['chat_history
```

```
--- Running Logic Test ---

--- Entering Agent Node ---

--- Deciding Next Step ---
Decision: END (Agent responded directly)
{'agent': {'chat_history': [HumanMessage(content='What is 2+2?', addition

Agent's Final Response (Logic Test): 2+2 equals 4.
```

```
import google.generativeai as genai
import os

# Ensure GOOGLE_API_KEY is loaded
if os.environ.get("GOOGLE_API_KEY") is None:
    from google.colab import userdata
    os.environ["GOOGLE_API_KEY"] = userdata.get('GOOGLE_API_KEY')
    if os.environ["GOOGLE_API_KEY"] is None:
        raise ValueError("GOOGLE_API_KEY not found in Colab secrets. Please ad

genai.configure(api_key=os.environ["GOOGLE_API_KEY"])
```

```
print("Listing available Gemini models with generateContent support:")
for m in genai.list_models():
  if "generateContent" in m.supported_generation_methods:
    print(m.name)
```

```
Listing available Gemini models with generateContent support:
models/gemini-2.5-flash
models/gemini-2.5-pro
models/gemini-2.0-flash
models/gemini-2.0-flash-001
models/gemini-2.0-flash-lite-001
models/gemini-2.0-flash-lite
models/gemini-2.5-flash-preview-tts
models/gemini-2.5-pro-preview-tts
models/gemma-4-26b-a4b-it
models/gemma-4-31b-it
models/gemini-flash-latest
models/gemini-flash-lite-latest
models/gemini-pro-latest
models/gemini-2.5-flash-lite
models/gemini-2.5-flash-image
models/gemini-3-pro-preview
models/gemini-3-flash-preview
models/gemini-3.1-pro-preview
models/gemini-3.1-pro-preview-customtools
models/gemini-3.1-flash-lite-preview
models/gemini-3.1-flash-lite
models/gemini-3-pro-image-preview
models/gemini-3-pro-image
models/nano-banana-pro-preview
models/gemini-3.1-flash-image-preview
models/gemini-3.1-flash-image
models/gemini-3.5-flash
models/lyria-3-clip-preview
models/lyria-3-pro-preview
models/gemini-3.1-flash-tts-preview
models/gemini-robotics-er-1.5-preview
models/gemini-robotics-er-1.6-preview
models/gemini-2.5-computer-use-preview-10-2025
models/antigravity-preview-05-2026
models/deep-research-max-preview-04-2026
models/deep-research-preview-04-2026
models/deep-research-pro-preview-12-2025
```

11. Propose a Scalable Business Use Case

Agentic RAG systems are highly valuable in document-heavy workflows. Let's discuss a practical enterprise application.

Use Case: Legal Contract Review

Problem: Legal firms spend immense hours manually reviewing complex contracts, legal documents, and case precedents to extract specific clauses, identify risks, ensure compliance, and answer client queries. This process is time-consuming, prone to human error, and requires highly specialized expertise.

Solution using Agentic RAG: An Agentic RAG system, similar to the one built here, can significantly automate and enhance this process:

1. **Document Ingestion & Indexing:** All legal contracts, policies, and case law documents are loaded, chunked, and embedded into a secure, persistent vector database (like ChromaDB).
2. **Agent-Powered Querying:** Lawyers can ask natural language questions (e.g., "What are the termination clauses in this contract?", "Does this agreement comply with GDPR Article 6?", "Show me all precedents related to intellectual property disputes for software companies.").
3. **Intelligent Routing:**
    - For simple, factual questions (e.g., "What is the effective date of Contract X?"), the agent directly retrieves the information without unnecessary database lookups.
    - For complex, knowledge-intensive questions, the agent intelligently uses the `knowledge_base_search` tool to retrieve relevant clauses, sections, or precedents from the vector store.
4. **Grounded Answers & Citation:** The agent is engineered to provide answers strictly grounded in the retrieved legal context. It can cite the specific document sections or pages from which the information was sourced, enhancing trustworthiness and allowing for quick verification.
5. **Risk Identification:** The agent could be prompted to identify clauses that deviate from standard templates or flag potential risks (e.g., "Are there any clauses in this contract that expose the client to unlimited liability?").

Benefits:

- **Reduced Human Overhead:** Automates the initial review and information retrieval, freeing up lawyers for higher-value analytical and advisory tasks.
- **Increased Accuracy & Consistency:** Reduces human error and ensures consistent application of legal knowledge across documents.
- **Faster Response Times:** Provides near-instantaneous answers to complex queries, dramatically speeding up contract review cycles.
- **Enhanced Compliance:** Helps ensure legal documents adhere to the latest regulations by quickly cross-referencing against updated legal databases.

- **Scalability:** Easily scales to handle vast volumes of documents without proportional increases in human resources.

This architecture transforms legal document review from a laborious, manual process into an efficient, AI-assisted workflow, allowing legal professionals to focus on strategic decision-making.

12. Visualize the LangGraph Execution Flow

Visualizing the graph helps understand the decision logic and routing transparency. We'll use `IPython.display` to render the graph.

```
    from IPython.display import Image, display

    try:
        display(Image(app.get_graph().draw_png()))
    except Exception as e:
        print(f"Could not visualize graph: {e}")
        print("Please ensure 'graphviz' is installed (e.g., !pip install pygraphvi

    print("Graph visualization displayed.")
```

README Structure for AgenticRAG-DTA

```
# AgenticRAG-DTA: Agentic RAG with LangGraph for Discovery-to-Action

A cutting-edge AI assistant combining Retrieval-Augmented Generation (RAG) w

## ✨ Features

*   **Intelligent Agentic Workflow:** Leverages LangGraph to orchestrate cor
*   **Retrieval-Augmented Generation (RAG):** Integrates a knowledge base se
*   **Dynamic Tool Use:** Agent intelligently decides whether to use a knowl
*   **Local Vector Store:** Uses ChromaDB for efficient storage and retrieva
*   **Flexible LLM Integration:** Designed to be adaptable to various LLMs (
*   **PDF Document Processing:** Loads, chunks, and embeds content from PDF

## 🛠️ How it Works

This project implements a Discovery-to-Action (DTA) framework using LangGrap

1.  **Document Ingestion:** A sample PDF document is loaded, split into mana
```

```
2.  **Vector Database (ChromaDB):** These embeddings are stored in a persis
3.  **Knowledge Base Search Tool:** A custom LangChain tool is defined to qu
4.  **Agentic Workflow (LangGraph):**
    *   An `AgentState` manages the conversation history and input/output.
    *   An `agent_node` uses an LLM (e.g., `gemini-2.5-flash`) to decide the
    *   A `tool_node` executes the `knowledge_base_search` tool if requested
    *   A `conditional_edge` dynamically routes the workflow based on the ag
    *   The entire workflow is compiled into a runnable agent.

## 🚀 Setup & Installation

Follow these steps to get the project running locally:

### Prerequisites

*   Python 3.9+
*   Google API Key (for Gemini models) added to Colab secrets as `GOOGLE_API

### Clone the Repository

```bash
git clone https://github.com/your-username/AgenticRAG-DTA.git
cd AgenticRAG-DTA
```

Install Dependencies

```
pip install -U langchain langchain-community langchain-chroma langgraph pypd
```

Run the Notebook

Open the `agentic_rag_dta.ipynb` notebook in Google Colab or your preferred Jupyter environment and execute the cells sequentially. Ensure your `GOOGLE_API_KEY` is configured in Colab secrets.

💡 Usage

Once the notebook cells are executed, the agent will be ready to answer questions. You can test it with:

- **Document-specific questions:** E.g., "What are the three main risks of bias mentioned in the document?"
- **General knowledge questions:** E.g., "What is 2+2?"
```

The agent will dynamically decide whether to use its `knowledge_base_search` tool based on the query.

📂 Project Structure

- `agentic_rag_dta.ipynb`: The main Jupyter notebook containing all the code and explanations.
- `ai_ethics.pdf`: Sample PDF document used for the knowledge base.
- `chroma_db/`: Persistent directory for the ChromaDB vector store.

🛣️ Future Enhancements

- Integrate more sophisticated decision-making for tool use.
- Add support for multiple tools and more complex tool orchestration.
- Implement a user interface for easier interaction.
- Expand document loaders to support various file formats.
- Fine-tune LLM for specific RAG tasks.

🤝 Contributing

Contributions are welcome! Please feel free to open issues or submit pull requests.

📄 License

This project is licensed under the MIT License - see the LICENSE file for details. ```